NANOWARE AI / PRACTICE SYSTEM

What Should I Practice?

A capability-first practice and project mission library for the ML Engineer Playbook

Practice is not a worksheet.
Every mission exists to turn knowledge into a capability. You should write code, make decisions, encounter failure, investigate it, and produce evidence that you can do the work.

The progression
Micro experiment → focused exercise → small build → broken build → realistic system → unfamiliar codebase → real issue → contribution → ambiguous engineering project

Do not practice everything.
Your AI mentor should select the next mission based on what you can already do, what you failed at, and what capability the next stage requires.

Practice rule: if the mission feels too easy, do not merely do more repetitions. Add ambiguity, constraints, failure modes, scale, or a new environment.

The Practice Operating System

The same mission can be used at different difficulty levels. The mentor should adapt it.

1 RECREATE	Can you reproduce the idea without copying the tutorial?
2 MODIFY	Can you change an assumption, input, architecture, or constraint?
3 BREAK	Can you intentionally make it fail and explain why?
4 DEBUG	Can you locate the failure systematically instead of guessing?
5 MEASURE	Can you define a metric and collect evidence?
6 DEFEND	Can you explain why your implementation/design is reasonable?
7 TRANSFER	Can you solve a different problem using the same underlying capability?

Evidence beats completion: a finished notebook is weak evidence. A tested implementation, benchmark, bug investigation, design explanation, reproducible repository, or defended trade-off is much stronger evidence.

Python Foundations

10 mission levels • Start where your capability is, then escalate difficulty

01 Micro	Write a CLI that accepts input, validates it, transforms it, and returns a useful result.
02 Core	Build a small text/data processing utility using functions, loops, dictionaries, lists and sets.
03 Files	Create a program that reads messy files, handles malformed records, and produces a clean output.
04 Errors	Design deliberate failure cases and build useful exception handling without hiding bugs.
05 Modules	Split a growing script into a small package with clear responsibilities.
06 Testing	Write tests for normal cases, edge cases and invalid input.
07 Debug	Give your mentor a failing program and diagnose it using a reproducible debugging process.
08 Build	Create a reusable data-ingestion CLI with configuration and logging.
09 Break	Inject bugs into the ingestion tool and recover from them without rewriting everything.
10 Proof	Explain the architecture, trade-offs, tests and failure modes to your mentor.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

Object-Oriented Python

10 mission levels • Start where your capability is, then escalate difficulty

01 Micro	Model a small real-world domain with classes and objects.
02 Core	Turn a procedural script into a maintainable object-oriented design.
03 Encapsulation	Design an object whose internal state cannot be corrupted through normal public usage.
04 Inheritance	Compare inheritance with composition using the same problem.
05 Polymorphism	Create interchangeable implementations behind one interface.
06 Abstraction	Define an interface for a component that may later have multiple implementations.
07 Design	Refactor a deliberately overcomplicated class into smaller responsibilities.
08 Testing	Test class behavior, invalid states and interactions.
09 Break	Introduce a state-management bug and trace it through object interactions.
10 Proof	Defend why you chose classes, composition or plain functions.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

Git & GitHub

10 mission levels • Start where your capability is, then escalate difficulty

01 Micro	Create a repository with meaningful commits instead of one giant commit.
02 Branching	Develop a feature on a branch and integrate it cleanly.
03 Conflict	Create and resolve a merge conflict while preserving intended behavior.
04 History	Use Git history to investigate when and why a regression appeared.
05 Rebase	Practice a rebase workflow on a small project.
06 Review	Perform a code review focused on correctness, tests and maintainability.
07 Issue	Turn a vague bug report into a reproducible GitHub issue.
08 PR	Submit a small pull request with tests and a concise explanation.
09 Break	Recover a repository after an intentionally bad change.
10 Proof	Explain your branch, commit and review strategy to your mentor.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

NumPy / Pandas / SQL / Data

10 mission levels • Start where your capability is, then escalate difficulty

01 NumPy	Implement vectorized numerical transformations and compare them with Python loops.
02 Arrays	Investigate shapes, broadcasting and indexing by constructing failing examples.
03 Pandas	Clean a deliberately messy dataset containing missing values, duplicate rows and bad types.
04 Join	Solve the same business question using different SQL join strategies.
05 SQL	Write aggregations, window functions and subqueries against a realistic dataset.
06 Validation	Create data-quality checks that reject invalid records.
07 EDA	Investigate an unfamiliar dataset and produce evidence-backed findings.
08 Performance	Measure a slow data operation and improve it without changing the result.
09 Break	Inject data leakage, type errors and malformed values; detect them automatically.
10 Proof	Deliver a reproducible data pipeline with validation and a short investigation report.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

Math for ML

10 mission levels • Start where your capability is, then escalate difficulty

01 Representation	Represent a small dataset as vectors and matrices; inspect dimensions programmatically.
02 Similarity	Compare vectors using distance, dot product and cosine similarity and interpret the differences.
03 Geometry	Visualize projections and investigate what information is lost.
04 Change	Numerically estimate a derivative and compare it with an analytical derivative.
05 Gradient	Visualize a simple loss surface and determine which direction reduces error.
06 Probability	Simulate random variables and compare empirical results with theoretical expectations.
07 Statistics	Estimate mean, variance, covariance and sampling behavior from repeated experiments.
08 Optimization	Implement gradient descent and investigate learning-rate failure modes.
09 Break	Make optimization diverge, stall or oscillate, then diagnose the cause.
10 Proof	Explain the mathematical meaning of a model parameter, gradient and objective using computation.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

Classical Machine Learning

10 mission levels • Start where your capability is, then escalate difficulty

01 Baseline	Build a simple baseline before training a sophisticated model.
02 Regression	Train a regression model and diagnose errors rather than only reporting a score.
03 Classification	Build a classifier and compare precision, recall, threshold and confusion matrix behavior.
04 Features	Create competing feature sets and measure whether each actually improves generalization.
05 Validation	Compare a single split with cross-validation and explain the difference.
06 Overfit	Construct a model that overfits deliberately and diagnose why.
07 Regularize	Compare model complexity with and without regularization.
08 Pipeline	Build a preprocessing + model pipeline that prevents leakage.
09 Experiment	Run controlled experiments where only one important factor changes at a time.
10 Proof	Defend model choice, metric, validation method and failure analysis.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

ML From First Principles

10 mission levels • Start where your capability is, then escalate difficulty

01 Linear Regression	Implement linear regression using only basic numerical operations.
02 Gradient Descent	Implement gradient descent and visualize convergence.
03 Logistic Regression	Build logistic regression from scratch and inspect probabilities.
04 Loss	Implement and compare different loss functions on controlled examples.
05 Backprop	Implement a tiny neural network and manually trace the forward/backward computation.
06 Decision Tree	Implement a small decision tree split criterion from first principles.
07 K-Means	Implement clustering and inspect sensitivity to initialization.
08 PCA	Implement a simplified PCA workflow and visualize dimensionality reduction.
09 Compare	Compare your implementation with a library implementation on identical data.
10 Proof	Explain where the library gains reliability, performance and engineering maturity.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

Deep Learning

10 mission levels • Start where your capability is, then escalate difficulty

01 Tensor	Build a tiny tensor-based computation and inspect shapes at every stage.
02 Training Loop	Write a complete training loop without relying on a high-level trainer.
03 Autograd	Investigate gradients by changing one operation and inspecting the result.
04 Optimization	Compare optimizers and learning rates on the same model.
05 CNN	Train a small CNN and inspect where errors concentrate.
06 Embeddings	Visualize learned representations and investigate what they encode.
07 Debug	Diagnose a model that is not learning by checking data, gradients, loss and architecture.
08 Overfit Test	Force a model to overfit a tiny dataset as a sanity check.
09 Break	Introduce a shape, normalization or label bug and identify it systematically.
10 Proof	Explain the complete training pipeline from data to learned parameters.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

PyTorch

10 mission levels • Start where your capability is, then escalate difficulty

01 Tensor Ops	Implement a numerical pipeline using tensors, indexing, broadcasting and device movement.
02 Dataset	Create a custom dataset and dataloader for a real dataset.
03 Model	Implement a small neural network as a reusable module.
04 Training	Write a transparent training/validation loop with metrics.
05 Checkpoint	Save, load and validate model checkpoints reproducibly.
06 GPU	Move a workload between CPU and GPU and measure the actual speed difference.
07 Debug	Use tensor shapes, gradients and hooks to diagnose a training bug.
08 Performance	Profile a training loop and identify the largest bottleneck.
09 Break	Create a silent device/dtype/shape bug and catch it with tests or assertions.
10 Proof	Explain your PyTorch architecture and why each training component exists.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

Transformers & LLMs

10 mission levels • Start where your capability is, then escalate difficulty

01 Tokenization	Inspect how text becomes tokens and construct examples where tokenization matters.
02 Attention	Implement a tiny attention mechanism and visualize attention weights.
03 Transformer	Build a minimal transformer block and trace tensor shapes.
04 Embeddings	Compare semantic similarity using embeddings on a small corpus.
05 Inference	Measure latency and memory for different inference settings.
06 Fine-tuning	Fine-tune a small model on a controlled dataset and evaluate it properly.
07 PEFT	Compare parameter-efficient adaptation with full fine-tuning conceptually and experimentally.
08 Evaluation	Create an evaluation set that exposes model failure modes instead of relying on one score.
09 Break	Construct prompts/data that reveal systematic weaknesses in the model.
10 Proof	Defend model, data, evaluation and inference trade-offs.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

RAG & Modern AI Systems

10 mission levels • Start where your capability is, then escalate difficulty

01 Retrieval	Build semantic retrieval over a small document collection.
02 Chunking	Compare chunking strategies and measure retrieval quality.
03 Embeddings	Test whether embedding similarity matches your intended notion of relevance.
04 Reranking	Compare retrieval before and after reranking.
05 Generation	Connect retrieved context to generation and inspect unsupported answers.
06 Grounding	Create tests that distinguish grounded from ungrounded responses.
07 Evaluation	Build a small RAG evaluation dataset with expected evidence.
08 Failure	Inject retrieval failures, missing context and conflicting documents.
09 Latency	Measure where latency occurs across ingestion, retrieval and generation.
10 Proof	Explain the full RAG system, its failure modes, evaluation and cost/latency trade-offs.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

Software Engineering for ML

10 mission levels • Start where your capability is, then escalate difficulty

01 Structure	Turn a notebook into a small production-style Python project.
02 API	Expose a model through a clean API with input validation.
03 Testing	Create unit, integration and regression tests for model-serving behavior.
04 Packaging	Package the project so another machine can install and run it.
05 Config	Move environment-specific settings out of source code.
06 Logging	Add structured logs that make a failed inference diagnosable.
07 Docker	Containerize the service and reproduce the environment.
08 CI	Run tests automatically on every change.
09 Review	Refactor the project after a simulated code review.
10 Proof	Give a technical walkthrough from request → code → model → response → failure handling.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

MLOps & Production

10 mission levels • Start where your capability is, then escalate difficulty

01 Serving	Deploy a model behind an API and measure real request behavior.
02 Tracking	Track experiments so you can reproduce which data/code/model produced a result.
03 Versioning	Version model and data artifacts and demonstrate rollback.
04 CI/CD	Create a pipeline that tests and packages the service automatically.
05 Monitoring	Define operational and model-quality signals worth monitoring.
06 Data Quality	Detect schema drift, missingness and invalid inputs before inference.
07 Latency	Measure p50/p95 latency and investigate the dominant bottleneck.
08 Cost	Estimate inference cost and compare two architecture choices.
09 Failure	Simulate a broken dependency or bad model deployment and recover safely.
10 Proof	Produce an operational runbook explaining deployment, monitoring and rollback.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

Open Source ML Engineering

10 mission levels • Start where your capability is, then escalate difficulty

01 Repository	Clone an unfamiliar ML repository and map its architecture.
02 Build	Get the project running locally and document every setup obstacle.
03 Tests	Understand the existing test structure and run the relevant subset.
04 Issue	Choose a real issue that matches your current capability.
05 Reproduce	Create the smallest reproducible example for the issue.
06 Investigate	Trace the bug from symptom to responsible code path.
07 Patch	Implement the smallest defensible fix.
08 Validate	Add or update tests and run targeted plus broader checks.
09 PR	Write a maintainer-friendly pull request explaining problem, cause, fix and tests.
10 Proof	Defend why your change is correct and what risks remain.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

ML Systems & Architecture

10 mission levels • Start where your capability is, then escalate difficulty

01 Architecture	Draw an end-to-end architecture for a model-powered application.
02 Data Flow	Trace data from ingestion to feature/model output and identify failure points.
03 Scale	Estimate what changes when traffic increases 10×.
04 Latency	Break total latency into components and identify optimization targets.
05 Reliability	Design retries, timeouts, fallbacks and graceful degradation.
06 Caching	Identify where caching helps and where it can create correctness problems.
07 Batch vs Online	Compare batch and online inference for the same business requirement.
08 GPU	Reason about memory, throughput and batching constraints.
09 Trade-offs	Compare two architectures using measurable criteria rather than preference.
10 Proof	Present the architecture and defend its technical and operational trade-offs.

Escalation: if you can complete the mission comfortably, change the data, remove instructions, introduce a failure, increase scale, or ask the mentor to give you an unfamiliar variant.

The Project Ladder

Projects should grow in ambiguity and engineering responsibility—not just in code size.

Level 1 — Utility	Build a useful Python tool. No ML required. Focus on inputs, outputs, errors, tests and packaging.
Level 2 — Data Product	Build a data-cleaning/analysis pipeline over messy real-world data with validation and reproducibility.
Level 3 — ML Product	Build a classical ML system with a baseline, experiments, evaluation, error analysis and an API.
Level 4 — From Scratch	Rebuild a core ML component from first principles and compare it with the established library.
Level 5 — Deep Learning	Train and debug a neural network system, including a real training loop and experiment tracking.
Level 6 — Modern AI	Build an evaluated RAG/LLM application with retrieval, grounding, failure analysis and latency measurements.
Level 7 — Production	Deploy a model-powered service with tests, packaging, containerization, monitoring and rollback thinking.
Level 8 — Real Codebase	Enter an unfamiliar open-source ML repository, reproduce an issue, patch it and submit a contribution.
Level 9 — Independent Engineer	Receive only an ambiguous problem statement. Research, design, build, evaluate, deploy and defend the system.

The Mentor's Job

The learner should not have to decide alone what to practice next. Give the mentor the current capability, recent attempts, failures and completed evidence. The mentor should choose the next mission and increase difficulty only when the evidence supports it.

Reusable prompt:
“You are my ML Engineer Practice Mentor. Based on what I have already learned and built, give me exactly one practical mission that targets my weakest important capability. Do not give me a step-by-step solution. First ask what I already know and what I would try. Make the mission progressively harder, include at least one realistic failure mode, and require evidence: code, tests, measurements, explanation or a reproducible artifact. When I finish, review my evidence and decide whether I should advance, repeat with a harder variant, or repair a gap.”